Assignment on GIT and Docker

Release date: June 8th
Submission date: 21st June (Sunday)
Corrected on: Marks for Q2 to be decided

Q1. **Git:** In this task we will get you acquainted with git. Git is a VCS (*Version Control System)* that lets you easily manage several versions of your code and collaborate with your teammates, no matter how small or large your code base is.

Total: 15 Marks

(checkout, add, commit, push, pull, branch, merge, rebase)

Part 1: First Commit: You (2 Marks)
Create a private GitHub repository using your GitHub account. Name it `<roll-no>-git`. If this name is unavailable, append random digits, for example `<roll-no>-git42`.

This question assumes that you have a friend who is working with you on this project. In a real-life collaborative setting, your friend would have a separate GitHub account, and you would add that account as a collaborator to your repository so that both of you can push changes to the same remote repository.

For this assignment, however, we will simplify the setup. You will act as both yourself and your friend by creating two separate local clones of the same remote repository. Configure one local clone with the Git author name `<roll-no>` and the other with the Git author name `<roll-no>-friend`. Both local repositories should point to the same GitHub remote. This setup logically serves the same purpose for learning without requiring a second GitHub account. Issue git instructions so that you have:

```
$:~/git-assignment/051046$ git config --local --list
...
remote.origin.url=git@github.com:amit23358/051046-git.git
...
user.name=051046
user.email=amit23358@gmail.com
```

For your friend:

```
$:~/git-assignment/051046-friend$ git config --local --list
...
```

```
remote.origin.url=git@github.com:amit23358/051046-git.git
...
user.name=051046-friend
user.email=amit23358@gmail.com
```

The following steps are to be performed by the 'you': (git clone, git add, git commit, git push)

1. Download **passwords.h** from the assignment resources folder. Create a file called **utils.h**. Implement a function with the following signature in it (don't forget to include **passwords.h** in it):
   ```
   bool login(string name, string password)
   ```
   This function returns true only if a user with the given name is present in the database(which is defined in **passwords.h**) and the password matches. Use auxiliary functions defined in **passwords.h** to complete this function.

2. Create a **main.cpp** that takes in two strings from **stdin**, a **name** and a **password**. Call the function **login** from main. If it returns true, print **Success!** else print **Login Failed :(**

3. Now add all the changes in your working directory to the **staging** area (`git add`) and commit these changes with the commit message **feat: Login API**

> This will create a **main branch** and your first **commit** in git. A commit is like a snapshot of your work. If you make any changes to your working directory or create any new commits, you can always come back to any previously created commit and ***checkout*** what you were up to when you made that commit.
>
> At its heart git is just a *content-addressable filesystem,* i.e. it stores files for you and you can retrieve any file, anytime, based on the content of that file. For doing that git internally calculates a ***hash*** for your file, and you can retrieve that file later by remembering the hash that git gives you. However, we don't do this dirty work ourselves. Git provides us with a nice set of *user-friendly* commands that hide away these details and make our lives easier.
>
> Now, use the command ***`git log`*** to see the *history* of commits that you have made. You will only see a single commit, showing a string of 40 characters, saying that your current HEAD is at *main*, as follows:
>
> ```
> commit 8e3d9f789c2595698f7f143556e8ad270109bbff (HEAD -> main)
> Author: 051046 <amit23358@gmail.com>
> Date:   Mon Jun 8 17:52:44 2026 +0530
>
>     feat: Login API
> ```

Git stores everything and anything that you throw at it(even your commits) in its *content-addressable filesystem*. Using this 40 character *hash value*, you can refer to any commit you have made. However, it's a pain to remember these long hash values when your project is very big. For this, git provides us with objects called **refs**.

Basically, git lets you attach names to some important commits / milestones in your project. So, to return/refer to a previous commit you can simply use this **name**. Several types of **refs** are available in git, the most frequently used are: **tags** and **branches**, and they behave differently.

Branch: Apart from other metadata, like the author, time, etc. for a commit, git also stores pointers to the commits that immediately came before it. A branch in a git is a movable pointer. As you make more commits on the current branch, this pointer moves forward and Marks to the latest commit.

Tag: a tag, as opposed to a ***branch***, is a static pointer, mostly used as a milestone to refer to important updates or mark different versions of your code.

All the files, log and git objects are stored inside ***.git*** folder inside the repo.

4. Create a file `README.md`. And report the **hash** of the last commit that you made in it. Add this file to your staging area and create a new commit with the message **Adding README**.

5. Push your commits to the remote branch **main** (`git push`)

Part 2: Branch off (Your Friend) (3 Marks)

Your friend thinks it'd be really cool if you assigned user-ids to all the users. But you don't buy it and ask your friend for a ***proof of concept***. Your friend is too lazy to write all the code from scratch. So, they decide to ***branch off*** from your current main branch and develop their proof of concept on this branch.

Your friend needs to perform the following steps:

1. Clone the repository and pull the latest commits (`git pull`)
    a. Create a branch named `thisisabetteridea`. And switch to this branch.
2. Download the file `new_passwords.h` from the resource folder and copy its contents to `passwords.h` in your current working directory.
3. You may or may not need to change the implementation of the function **login** in `utils.h`.
4. Add changes to the staging area and commit your work with the commit message **feat: The Better Idea**
5. Push your commit to the remote branch `thisisabetteridea`

Part 3: Fixes (You) (2 Marks)

Your client is complaining that their users are confused and don't know what to do when they land on the login page. You realize that you forgot to prompt users asking for their names and passwords.

You need to do the following:

1. Change the `main` function in main.cpp, ask the user to **Enter Name:** and **Enter Password:** before reading the two strings from stdin.
2. Add your changes to the staging area and commit your work with the commit message **Fix: Adding Login Prompts**
3. Push your commits to the remote branch main

Part 4: Rebase / Merge (Your Friend) (4 Marks)

Your friend finds out that you have made changes to the *main* branch after they *branched off*. Now, to test their code, your friend needs all the changes you made in their branch as well.

Your Friend can do one of the following things: (git rebase, git merge). Let us stay with git rebase. .
Rebase their branch (***thisisabetteridea****)* to your *main* branch.

For this your friend needs to:

1. Checkout to their branch ***thisisabetteridea***
2. Use the appropriate git command to rebase to your *main* branch
3. Resolve conflicts if any and continue to rebase **if needed**
   (git rebase --continue)
4. Push their latest commits to the remote branch ***thisisabetteridea***

Part 5: (You) (4 Marks)

After your friend implements the proof of concept, you finally agree to agree. Now, you need to merge all their work into your *main* branch which you are using to deploy the login page.

For this, you need to do the following things:

1. Checkout your main branch
2. Use the appropriate commands to merge his ***thisisabetteridea*** branch into your main branch.
3. Push your latest commits to the *remote main* branch

That's it, you completed the basic login page your client asked for.

Submission: First run the command `git log --oneline --graph --all --decorate` from your repository and redirect the result to `q1/my-log.txt`. Run the same command from your friends repo and redirect the result to `q1/friend-log.txt`. The graph like structures that you just recorded are the history of all the branches and commits.

Now what are the instructions that you ran from your and your friends repos to get logs above? Record these in `git-history.sh`. The way we shall be checking the program is to first run `git-history.sh` and ascertain that one indeed gets the logs that you submitted. Then we shall check for the correctness of the logs.

```
q1
├── git-history.sh
|---my-log.txt
`---friend-log.txt
```

Q2. In this assignment, you will build a small multi-container application using Docker Compose. The application must contain two services: an API service named **`api`** and a tester service named **`tester`**.

The **`api`** service will run a Python HTTP server that provides a few fixed endpoints. The **`tester`** service will wait until the **`api`** service is healthy, contact it using the Docker Compose service name **`api`**, check the responses, and print exact output. This assignment tests your understanding of Docker Compose services, Dockerfiles, health checks, inter-container networking, and service startup ordering.

Your submission for q2 must have exactly the following structure:

```
q2
├── docker-compose.yml
├── api
│   ├── Dockerfile
│   └── server.py
└── tester
    ├── Dockerfile
    └── test_api.py
```

The file and directory names must match exactly.

Services

Your **`docker-compose.yml`** file must define exactly two services: **`api`** and **`tester`**. The **`api`** service must be built from the **`api`** directory. The tester service must be built from the **`tester`** directory. The **`tester`** service must contact the API using the hostname **`api`**, not localhost, not **`127.0.0.1`**, and not the host machine's IP address.

API Service Requirements

The **`api`** service must run a Python HTTP server listening on **`0.0.0.0:5000`**. It must provide the following **`HTTP`** endpoints.

Endpoint 1: **`GET /health`**
This endpoint must return **`HTTP`** status code **`200`** and the exact response body: **`OK`**

Endpoint 2: **`GET /square/7`**
This endpoint must return **`HTTP`** status code **`200`** and the exact response body: **`49`**

Endpoint 3: **`GET /reverse/docker-compose`**.
This endpoint must return **`HTTP`** status code 200 and the exact response body: **`esopmoc-rekcod`**

Endpoint 4: **`GET /sum?x=13&y=29`**
This endpoint must return **`HTTP`** status code **`200`** and the exact response body: **`42`**

You may implement the API using only the Python standard library.

Health Check Requirement

The `api` service must define a Docker Compose health check. The health check must test whether: `http://127.0.0.1:5000/health` returns the expected result. The `tester` service must use Docker Compose startup ordering so that it starts only after the `api` service is healthy. A fixed delay such as `sleep 10` is not an acceptable substitute for a health check.

Tester Service Requirements

The `tester` service must run the script `test_api.py`. This script must send HTTP requests to the `api` service using the base `URL: http://api:5000` The tester must check all four endpoints. If all tests pass, the tester must print exactly the following five lines, in this order:

```
HEALTH=OK
SQUARE=49
REVERSE=esopmoc-rekcod
SUM=42
ALL_TESTS_PASSED
```

The tester must then exit with status code `0`. If any endpoint gives an incorrect response, the tester must exit with a non-zero status code.

Dockerfile Requirements

The `api/Dockerfile` must build an image that runs `server.py`. The `tester/Dockerfile` must build an image that runs `test_api.py`. Both `Dockerfile`s should use a Python base image such as: `python:3.10-slim`. Each `Dockerfile` must set an appropriate working directory using `WORKDIR`. Each Dockerfile must copy only the required files into the image.

Compose Requirements

The file `docker-compose.yml` must:

1. define the services `api` and `tester`;
2. build each service from its corresponding directory;
3. define a health check for `api`;
4. ensure that `tester` starts only after `api` is healthy;
5. allow `tester` to reach api using the service name `api`.

You do not need to publish the API port to the host machine. The tester should communicate with the API through the private Docker Compose network.

How the Assignment Will Be Tested

The grader may run commands similar to the following:

```
docker compose down --volumes --remove-orphans
docker compose up --build --abort-on-container-exit --exit-code-from tester
docker compose logs tester
docker compose down --volumes --remove-orphans
```

The grader will check that the tester logs contain the exact expected output:

```
HEALTH=OK
SQUARE=49
REVERSE=esopmoc-rekcod
SUM=42
ALL_TESTS_PASSED
```

The grader may also inspect **`docker-compose.yml`** to check that api has a health check and that **`tester`** depends on the healthy **`api`** service.

Submission guideline:

Create the following directory structure, zip and submit.

```
<roll-no>
├── q1
│   ├── git-history.sh
│   └── ...
│
└── q2
    ├── docker-compose.yml
    └── ...
```

Acknowledgements: Rushabh Kanadiya, TA 2019 batch for the first question.